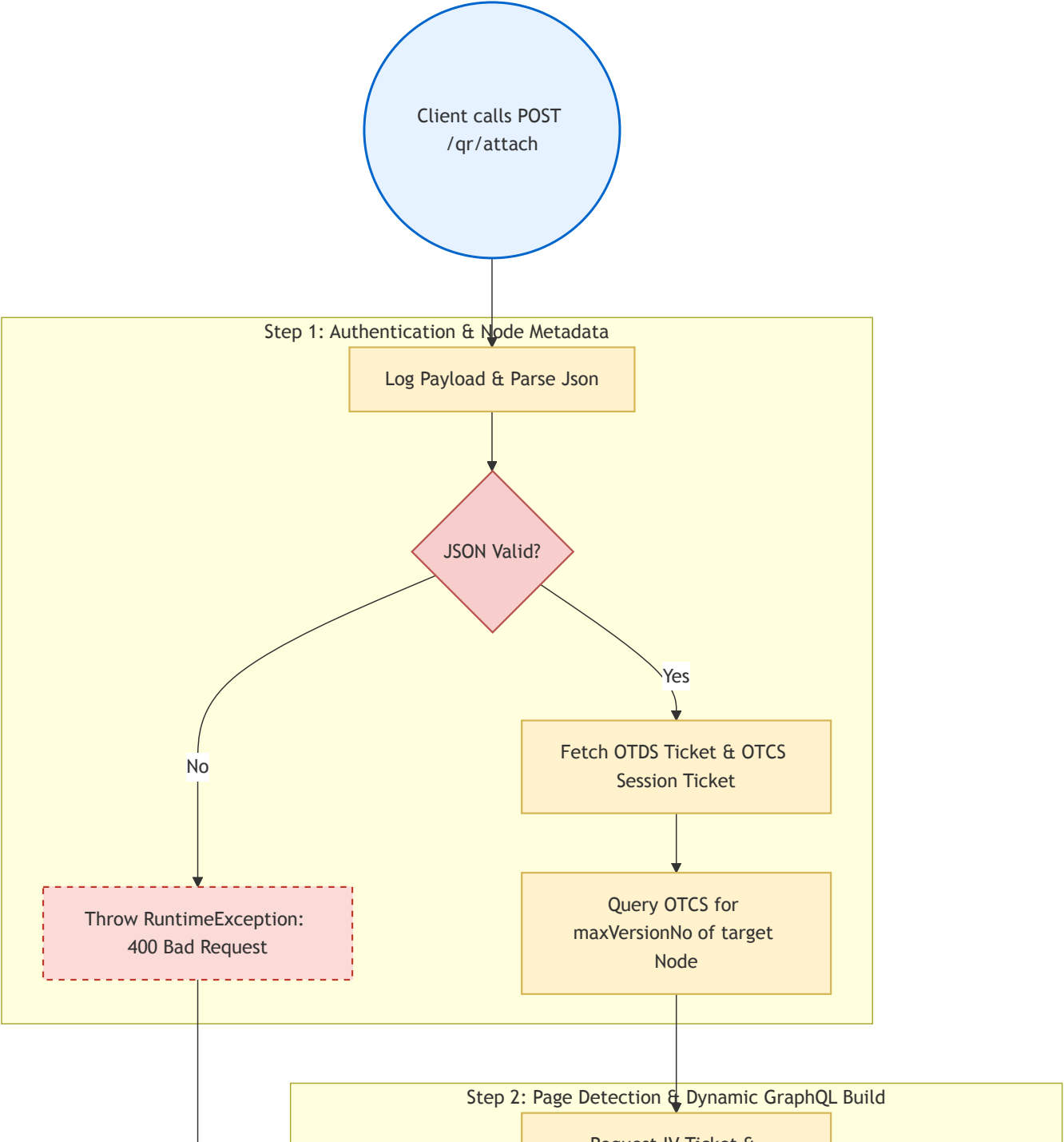


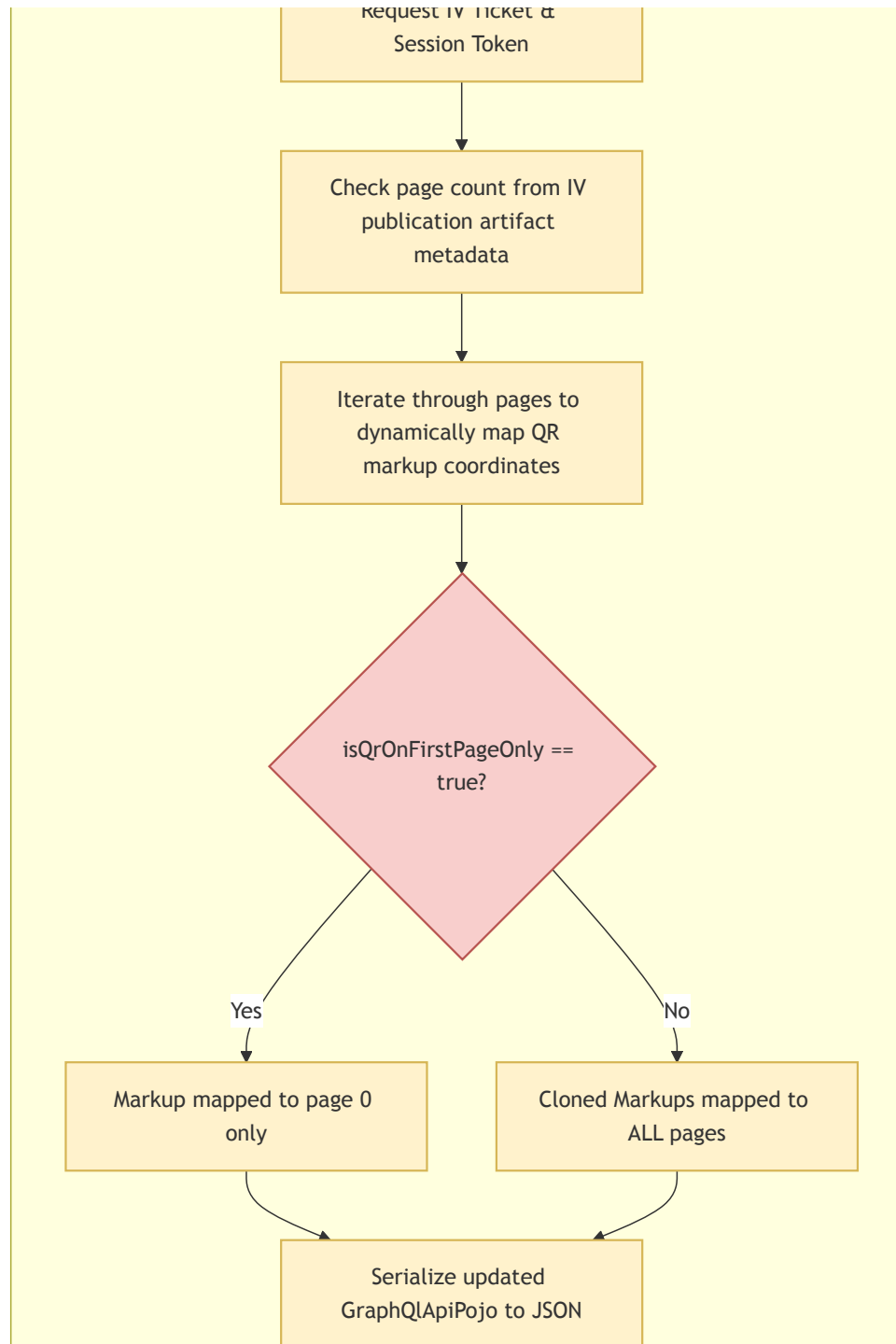
Operational Flow & Exception Handling: /qr/attach

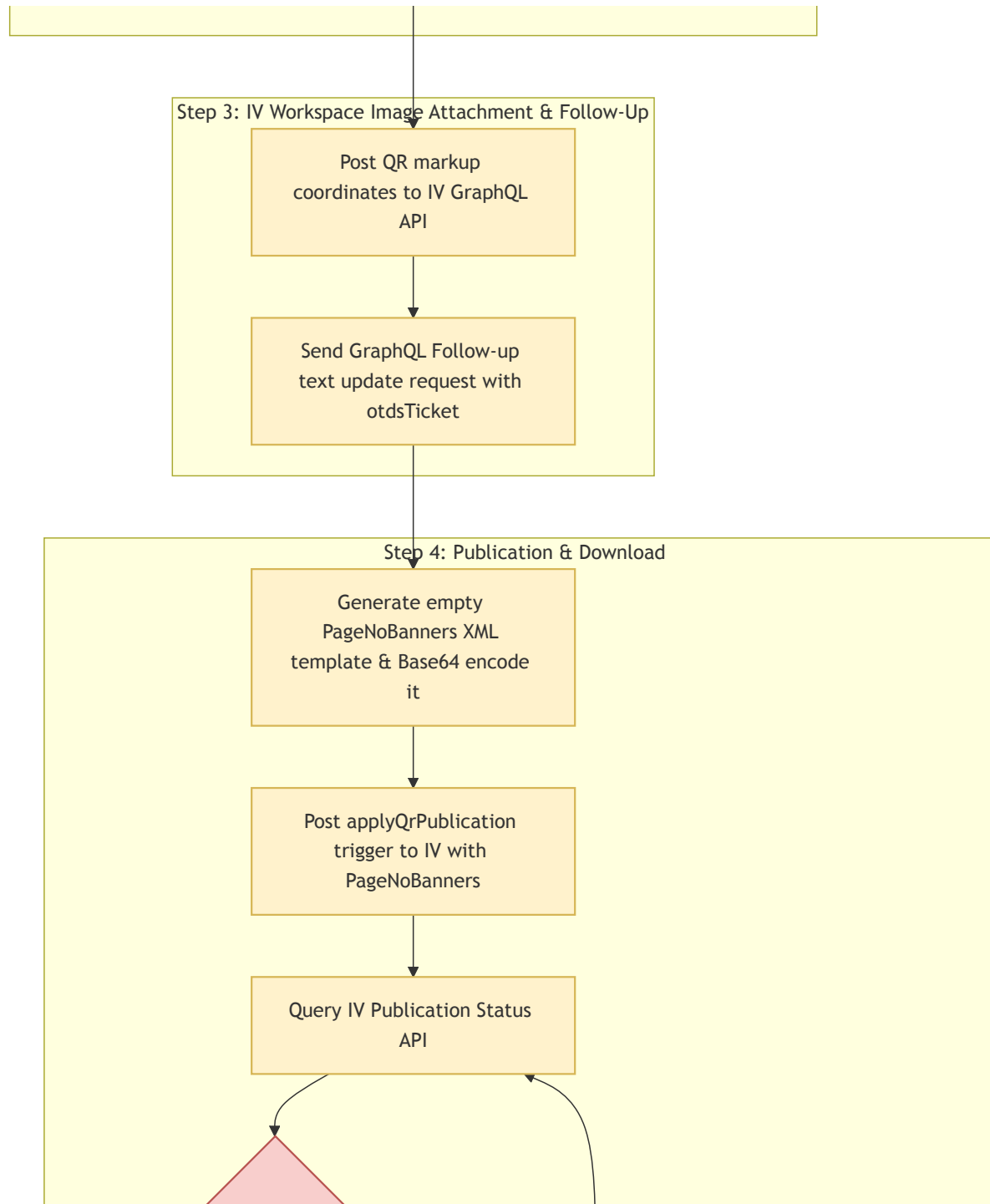
This document details the complete end-to-end execution flow and exception propagation system for the /qr/attach endpoint.

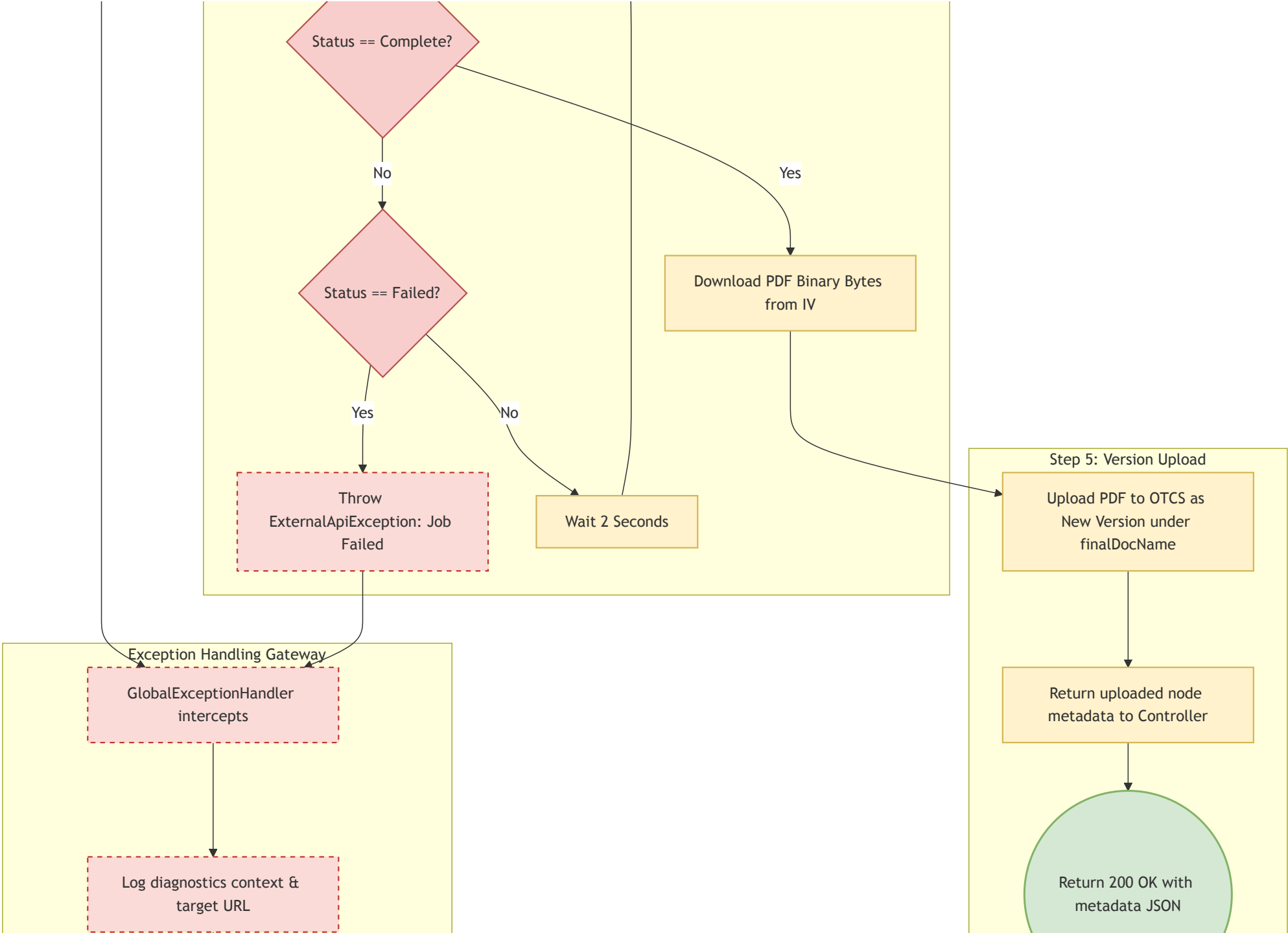
1. Process Flow Diagram (Boxes & Arrows)

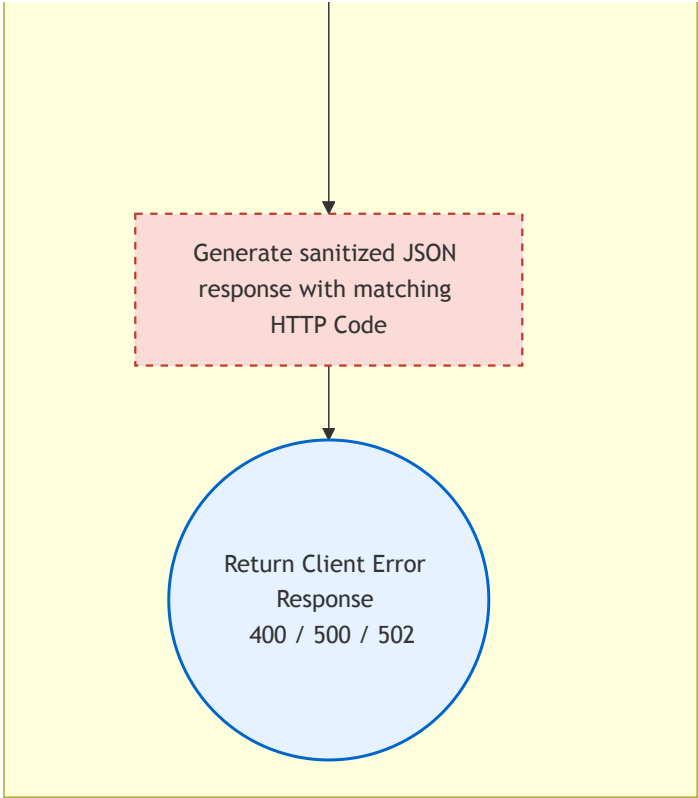
This flowchart traces the step-by-step process of the /qr/attach endpoint, highlighting decision gates (diamonds), logical operations (rectangles), and the execution path.



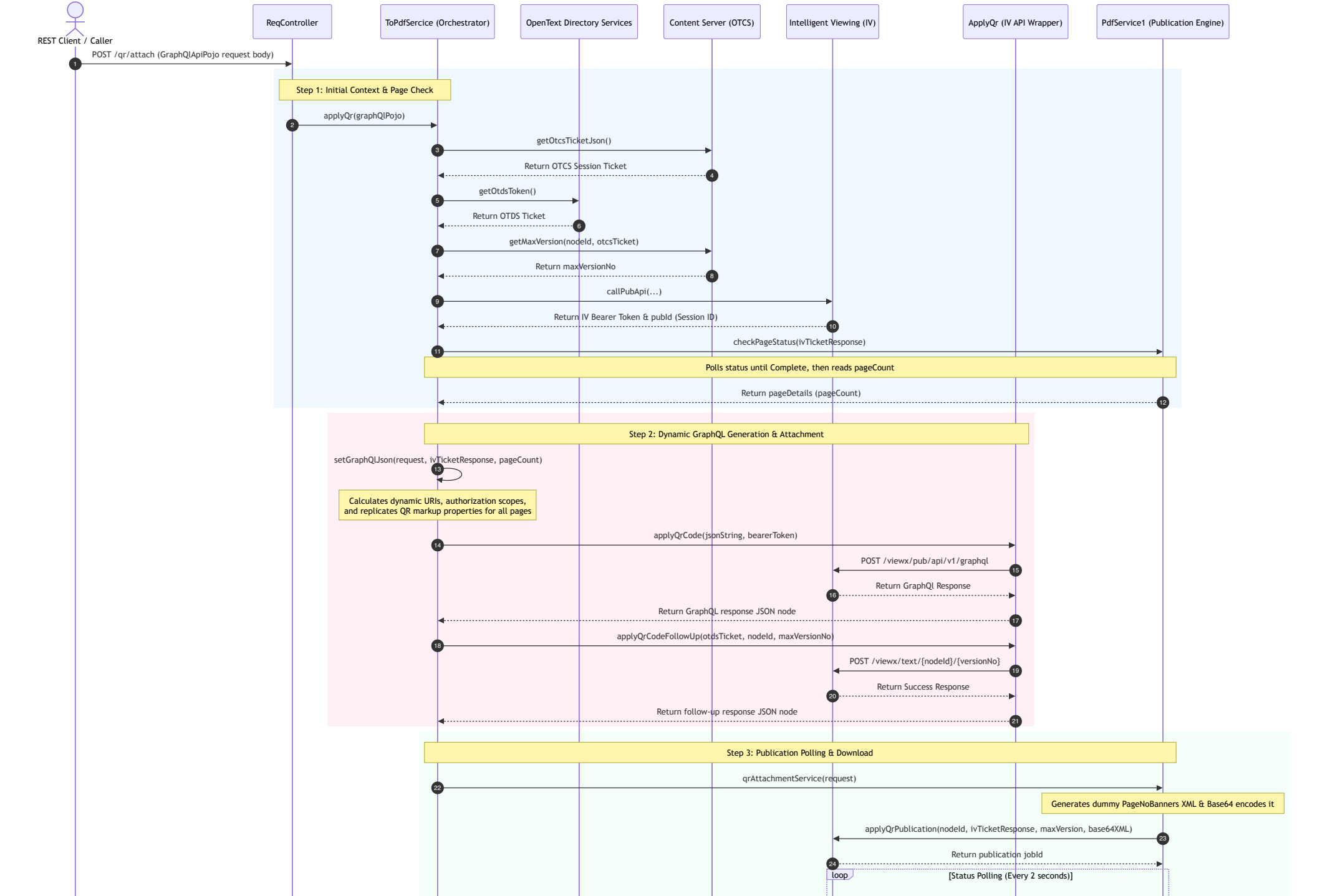


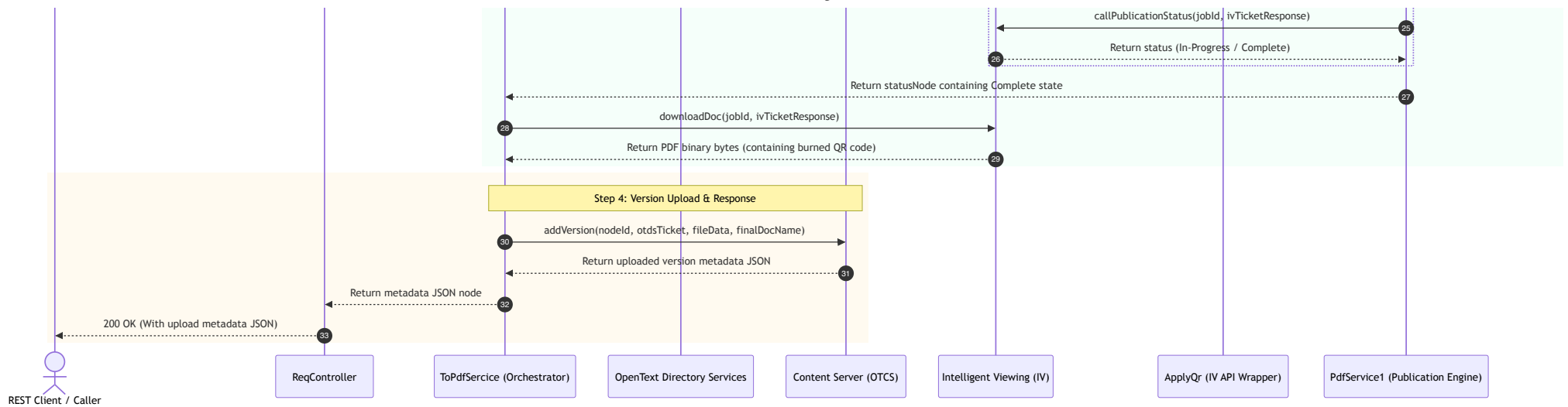






2. Happy Path Sequence Diagram





3. Step-by-Step Execution Mechanics

1. Entry Point (ReqController.java#attachQr):

- Exposes an HTTP POST mapping at /qr/attach .
- Receives a GraphQLApiPojo body specifying the document target, the QR code image source node, and layout details.
- Delegates execution to toPdfService.applyQr(graphQLPojo) .

2. Authentication & Metadata Retrieval (ToPdfService.java#applyQr):

- Authenticates using otdsToken.getOtdsTicketJson() and otdsToken.getOtdsToken() to retrieve tickets.
- Fetches the highest current document version of the target node in Content Server via allVersions.getMaxVersion .
- Obtains a rendering session ticket and bearer token from Intelligent Viewing (IV) using ivTicket.callPubApi .

3. Page Count Extraction (ToPdfService.java#applyQr):

- Invokes checkPageStatus which polls until the document page analysis is ready.
- Reads the exact pageCount of the document dynamically from the artifact content JSON structure.

4. Dynamic Markup Mapping (ToPdfService.java#setGraphQLJson):

- Reads user ID, constructs the QR code URI (/nodes/{imgNodeId}/versions/{imgVersion}/content), and extracts publication details.

- Checks if `isQrOnFirstPageOnly` is true. If yes, it maps the QR markup payload exclusively to page `0`.
- Otherwise, it replicates/clones the markup coordinates across all pages of the document (`0` to `pageCount - 1`).

5. IV Workspace Update (`ApplyQr.java`):

- Invokes `applyQrCode` to POST the JSON payload to the IV GraphQL endpoint (`/api/v1/graphql`).
- Invokes `applyQrCodeFollowUp` to send a POST request with the placeholder text payload to `/api/v1/viewx/text/{nodeId}/{versionNo}` to commit the layout modification inside the IV workspace.

6. Publication Pipeline (`PdfService1.java#qrAttachmentService`):

- Since no visual text banner is required for this operation, the service generates a placeholder `PageNoBanners` XML template configured for 0 pages, converts it to base64, and initiates the publication process in IV via `publication.applyQrPublication`.
- Polls the job status every 2 seconds via `checkStatus`.

7. Download & Upload Version (`ToPdfService.java#applyQr`):

- Once complete, it downloads the compiled PDF document bytes from the IV API containing the burned QR code.
- Commits the updated PDF back to the Content Server as a new version under the name specified by `finalDocName` via `addVersion.addVersion`.

4. Exception Handling & Propagation Details

Downstream Error Translation Flow

1. Downstream API Failures:

- OKHttp clients intercept HTTP responses. If a request is not successful (status code is not in the 2xx range), the code extracts the raw error body.

2. Exception Construction:

- An `ExternalApiException` is raised containing the status code, contextual action name (e.g. "QR Attachment"), and the targeted URL.

3. Global Advice Mapping (`GlobalExceptionHandler.java`):

- The `@ExceptionHandler` catches the `ExternalApiException`, formats a sanitized JSON response payload indicating which phase (such as OTCS upload, IV GraphQL attachment, or status polling) failed, and responds to the REST client with the corresponding HTTP code.